

Loran — Product Requirements Document v0.1

Field	Value
Project	Loran
Tagline	The Steelbore reference manual.
Document	Product Requirements Document (PRD)
Version	0.1.0 (initial draft)
Date	2026-05-11
Author	Mohamed Hammad
Maintainer	Mohamed Hammad Mohamed.Hammad@Steelbore.com
Copyright	(c) 2026 Mohamed Hammad
License	GPL-3.0-or-later
Website	https://Loran.Steelbore.com/
Governed by	Steelbore Standard v1.1, Steelbore SFRS v1.0.0
Spec	<code>loran-spec-v0_2.md</code> (canonical technical reference)

Table of Contents

1. Executive Summary
2. Background & Problem Statement
3. Vision & Strategic Position
4. Target Users & Personas
5. Goals
6. Non-Goals (Explicit Exclusions)
7. User Stories
8. Functional Requirements
9. Non-Functional Requirements
10. Product Surface Overview
11. Page Authoring Workflow

12. Distribution & Trust Model
 13. Steelbore Ecosystem Integration
 14. Phasing & Release Plan
 15. Success Metrics
 16. Dependencies
 17. Risks & Mitigations
 18. Open Questions
 19. Steelbore Standard v1.1 Compliance Audit
 20. References & Related Documents
-

1. Executive Summary

Loran is the canonical, agent-friendly reference tool for Steelbore-based systems. It is to Steelbore what `man` is to Unix and `info` is to GNU: a system-level handbook for every tool the system ships and recommends — with one critical difference, it is agent-native (`--json`, `schema`, MCP) from day one.

The product answers three questions about the tool catalog of a Steelbore system:

1. **What tools are available here?** — categorised browse of the curated catalog.
2. **What does this tool do, and what does it replace?** — Steelbore-curated intro, with `tldr` fallback.
3. **What replaces the legacy tool I know?** — reverse lookup (`loran find ls`) → `eza`).

A separate verb (`loran help <tool>`) captures live `--help` output from any binary on `$PATH`, rendered in a deliberately de-themed frame so curated content stays visually distinct from uncurated passthroughs.

Loran ships in three phases: **Ingot** (text-mode catalog + JSON + bundled pages), **Billet** (TUI + signed tarball updates + overlays + page authoring — the 1.0 milestone), and **Bloom** (read-only MCP surface + JSON Schema for agent function-calling + auto-ingestion of SFRS `describe` from other Steelbore CLIs).

The detailed technical specification — workspace layout, dependency stack, resolution chains, page format, JSON envelope, exit codes, file system layout — lives in `loran-spec-v0_2.md`. This PRD defines *what Loran must accomplish*; the spec defines *how*.

2. Background & Problem Statement

2.1 The world without Loran

A user installing a fresh Steelbore distro (Bravais today, Ferrite OS tomorrow) inherits ~150 tools, many of which are Rust-native replacements for legacy Unix utilities. Their existing knowledge

maps onto a different toolchain: `ls`, `cat`, `grep`, `ps`, `find`, `top`, `cp`, `dig`, `curl`. The Steelbore equivalents — `eza`, `bat`, `rg`, `procs`, `fd`, `bottom`, `xcp`, `dog`, `xh` — solve the same problems differently, with different flags, different defaults, and different invariants.

Today, that user has to assemble their orientation from disparate sources:

- `man` pages are often missing or thin for modern Rust tools (many ship only `--help` text).
- `tldr-pages` are excellent but generic, opinionated about a global community consensus rather than Steelbore's particular curation.
- **Blog posts and READMEs** are scattered, undated, and hard to discover from the terminal.
- `<tool> --help` is authoritative but uncured, formatted inconsistently across tools.
- **Steelbore-specific opinions** ("`eza` is our default; alias `ls=eza` in Nushell; pairs well with `bat`") exist only as tribal knowledge.

The result: even after installing a Steelbore system, a user spends days or weeks discovering which tool does what, which is safe to alias, and which pair well together. AI agents working on the same systems face the same problem amplified — they cannot rely on training-data familiarity with cutting-edge Rust tools, and they have no structured way to query what's available locally.

2.2 The specific gaps Loran addresses

Gap	Today's Workaround	Loran's Answer
No structured catalog of Steelbore-recommended tools	Read the distro README or trawl <code>/usr/bin</code>	<code>loran list</code> , <code>loran categories</code> , TUI browse
No way to ask "what replaces X?"	Search engine, hope for a good blog post	<code>loran find ls</code> → <code>eza</code>
No Steelbore-curated commentary alongside upstream docs	Wiki or scattered docs	Curated pages overlayed on top of tldr / <code>--help</code>
No machine-readable catalog for agents	Agent guesses from training data	<code>--json</code> envelope + MCP surface
No clear "this is safe to alias" signal	User reads release notes carefully	<code>safe_alias_for</code> frontmatter field
No companion-tool recommendations	Word of mouth	<code>pairs_with</code> frontmatter field
No way for the upstream catalog and the user's own notes to coexist	Edit upstream files in place	Per-distro and per-user overlay layers

2.3 Why now

Three factors make this the right moment:

1. **The Steelbore tool catalog has reached critical mass.** Bravais ships ~150 packages by default; further distros (Ferrite OS) are imminent. Without a curated catalog, that breadth becomes navigation debt.
 2. **AI-agent CLIs have become primary users of Unix tooling.** Steelbore's SFRS v1.0.0 codifies this by mandating `--json`, `schema`, and MCP on every Steelbore CLI. A system-wide catalog that exposes the same agent surface unifies the ecosystem.
 3. **tldr-pages proved the tarball + Markdown model works at scale.** Loran can reuse the distribution pattern while overlaying Steelbore's curation on top, avoiding both the maintenance burden of forking tldr and the limitations of being only a tldr client.
-

3. Vision & Strategic Position

3.1 Vision

A Steelbore user — human or agent — should be able to answer any "what tool should I use for X?" or "what is X?" question by typing one short command, in under a second, without leaving the terminal, with results that reflect Steelbore's deliberate curation rather than generic consensus.

3.2 Strategic position within Steelbore

Loran sits at a particular intersection in the Steelbore ecosystem:

- **For users:** the *first command* a new Bravais or Ferrite OS user runs after installation. The orientation layer.
- **For curators (i.e. the maintainer and contributors):** the *canonical place* to encode Steelbore's tool opinions. If it's not in Loran, it's not Steelbore-endorsed.
- **For agents:** the *primary tool-discovery surface* for any AI agent operating on a Steelbore system.
- **For other Steelbore CLIs:** the *ambient reference layer* — every Steelbore CLI's `--help` and error messages can point users to `loran show <self>` for fuller context.

Loran is not a competitor to `man`, `info`, or `tldr`; it is the curation layer above them, with its own content and a structured agent surface.

3.3 What Loran is *not*

To prevent scope drift, Loran is explicitly not:

- A package manager (that's Craton).
- A shell or shell framework (that's the user's choice, though the Bravais default profile integrates Loran-recommended aliases).

- A documentation rendering engine for arbitrary projects (it renders its own pages and captures live `--help`; no general-purpose man-page or info-renderer mode).
 - A tool installer or version manager (catalog entries describe tools, they don't install them).
 - A real-time system monitor (`bottom` does that; Loran tells you `bottom` exists).
-

4. Target Users & Personas

4.1 Primary personas

P1 — The Steelbore newcomer (human, terminal-fluent). Just installed Bravais or Ferrite OS. Comfortable with traditional Linux/BSD tools (`ls`, `cat`, `grep`). Wants to know what's different about this distro and which modern tools to learn. Reads in English, uses Nushell or Ion by default. Will spend the first hour exploring the system; Loran is the obvious starting point.

P2 — The AI agent operating on a Steelbore system. Could be Claude Code, Codex CLI, Gemini CLI, Cursor agent, or any other LLM-driven tool that invokes commands on the user's behalf. Needs to discover what tools are available locally and which ones to use for a given task. Must be able to call Loran without TUI activation, parse structured JSON output, and rely on stable schemas. Has a finite context-window budget so terseness matters.

P3 — The seasoned Steelbore user (human, daily driver). Has used Bravais/Ferrite OS for months; knows most of the catalog. Uses Loran occasionally — to look up a tool they've forgotten, to remind themselves of a flag, to onboard a colleague. Wants Loran to be fast (sub-50ms cold) and out of the way.

4.2 Secondary personas

P4 — The sysadmin deploying Steelbore at scale. Sets up multiple Bravais or Ferrite OS machines. Cares about reproducibility (every machine has the same Loran catalog), updateability (catalog refreshes from a signed source), and trust (no surprises in what content gets installed).

P5 — The tool author (Steelbore or external). Wants their tool added to the Loran catalog. Needs a clear authoring workflow (`loran new`), a documented frontmatter schema, and a path to contribute upstream.

4.3 Tertiary personas

P6 — The accessibility-dependent user. Uses a screen reader, prefers high-contrast text mode, may have motor-control needs that benefit from keyboard-only navigation. Loran must work for them as a first-class concern, not an afterthought.

5. Goals

The non-negotiable outcomes Loran must achieve to be considered successful.

5.1 Product goals

ID	Goal
G-01	A user can list, browse, and search the Steelbore tool catalog in a TTY without any prior knowledge of which tools are catalogued.
G-02	A user can reverse-look-up any legacy tool name (<code>ls</code> , <code>cat</code> , <code>grep</code> , etc.) and receive Steelbore's modern recommendation.
G-03	A user can see, for any catalogued tool, whether it is safe to alias to a legacy name (<code>safe_alias_for</code> field).
G-04	A user can see, for any catalogued tool, recommended companion tools (<code>pairs_with</code> field).
G-05	An AI agent can discover the full tool catalog via <code>--json</code> output or MCP without the TUI ever activating.
G-06	The catalog can be refreshed via a single command from a cryptographically signed upstream source.
G-07	The catalog supports three layers of overlay: upstream Steelbore curation, per-distro overlays, and per-user customisations.
G-08	A curator (the maintainer, contributors) can author new pages with a structured workflow that prevents schema drift.
G-09	Other Steelbore CLIs can integrate Loran by referencing <code>loran show <self></code> in their <code>--help</code> output.
G-10	Loran exposes its own data model via <code>loran schema</code> (JSON Schema Draft 2020-12) for agent function-calling.

5.2 Quality goals

ID	Goal
Q-01	Sub-50ms cold-cache response time for <code>loran show <known-tool></code> .
Q-02	100% Rust implementation; no <code>unsafe</code> blocks outside well-justified FFI.
Q-03	POSIX-compliant default output (parseable with <code>grep</code> / <code>awk</code> / <code>cut</code> / <code>sed</code> / <code>tr</code> alone).

ID	Goal
Q-04	WCAG 2.1 AA contrast in every UI surface.
Q-05	Zero telemetry, zero analytics, zero network calls except <code>loran update</code> .
Q-06	Atomic tarball updates: a failed verify never corrupts the working catalog.
Q-07	Predictable failure modes: every error code has an actionable hint following SFRS tips-thinking discipline.

6. Non-Goals (Explicit Exclusions)

Items deliberately excluded from scope. Listed up front to prevent scope creep.

ID	Non-Goal
NG-01	Loran does not install or update tools. Package management is Craton's responsibility.
NG-02	Loran does not author tldr pages. It consumes them as a fallback when no curated Steelbore page exists.
NG-03	Loran does not provide a GUI. Terminal-first by design. A future GUI is not a v1 concern.
NG-04	Loran does not embed a scripting language or plugin system. Pages are static Markdown + TOML; no eval, no execute.
NG-05	Loran does not write to the upstream <code>pages/</code> tree at runtime. That directory is sync target only; user edits live in overlays.
NG-06	Loran does not invoke arbitrary binaries via MCP. The MCP surface is strictly read-only; <code>loran help</code> is human-mode only.
NG-07	Loran does not provide real-time tool discovery (e.g., scanning <code>/usr/bin</code>). Phase 3 may auto-ingest from SFRS <code>describe</code> , but only for binaries already known to be Steelbore-native.
NG-08	Loran does not localise the CLI surface in v1. English-only for command names, flag descriptions, error messages. Page content i18n is a v1.x concern.
NG-09	Loran does not host or distribute upstream pages itself. A separate publisher pipeline (out of scope for this PRD) produces the signed tarball.
NG-10	Loran does not provide a web-based catalog viewer. A future static-site export of pages is conceivable but not in scope for v1.

7. User Stories

Concrete scenarios that drive the requirements. Format: `As a <persona>, I want <action>, so that <outcome>.`

7.1 First-run discovery

- US-001.** As P1 (newcomer), I want to run `loran` after installing Bravais and immediately see a categorised browse of every tool the distro recommends, so that I can orient myself without searching the web.
- US-002.** As P1, I want `loran show eza` to render a Steelbore-curated explanation of eza — including how it's aliased in the default Bravais Nushell profile — so that I learn the distro's idioms, not just the tool's upstream docs.

- **US-003.** As P1, I want `(loran find ls)` to tell me that eza is the Steelbore replacement, and to indicate whether `(alias ls=eza)` is safe (it isn't, fully) or risky (it is, partially), so that I make informed shell-rc decisions.

7.2 Daily reference

- **US-010.** As P3 (daily driver), I want `(loran show <tool>)` to complete in under 50ms so that it does not interrupt my thought process.
- **US-011.** As P3, I want to fuzzy-search the catalog `(loran search filesystem)` so that I can find tools by problem domain even when I forget the binary name.
- **US-012.** As P3, I want to look up `(pairs_with)` recommendations so that I discover complementary tools I had not learned about.

7.3 Live --help capture

- **US-020.** As P3, I want `(loran help <tool>)` to capture and render the tool's own `(--help)` output when I just want flag reference, so that I get authoritative upstream documentation without leaving the catalog UX.
- **US-021.** As P3, I want the live-capture frame to be visually distinct from curated content (monochrome, "LIVE OUTPUT" header), so that I never confuse uncurated upstream text with Steelbore-endorsed commentary.
- **US-022.** As P3, I want the live capture to fail gracefully (clear error, runnable hint) if the binary is absent or hangs, so that I am never left wondering whether it worked.

7.4 Authoring

- **US-030.** As P5 (tool author), I want `(loran new widgetctl)` to scaffold a new page with all required frontmatter fields prefilled or prompted-for, so that I cannot accidentally produce an invalid page.
- **US-031.** As P5, I want `(loran new --no-edit --category=... --replaces=... ...)` to be fully scriptable, so that I can generate pages from CI or a batch tool.
- **US-032.** As P5, I want `(loran validate)` to surface schema violations with file paths and line numbers, so that I can fix problems before committing.
- **US-033.** As P5, I want a `(--scope=upstream)` flag that writes into a local checkout of the upstream pages tree, so that I can prepare a contribution PR without manually copying files.

7.5 Agent workflows

- **US-040.** As P2 (AI agent), I want `(loran list --json)` to return a stable, schema-documented inventory of every tool, so that I can ground my recommendations in what the user actually has installed.
- **US-041.** As P2, I want `(loran show <tool> --json)` to distinguish curated body content from tldr fallback (via `(body.kind)`), so that I can convey provenance to the user.

- **US-042.** As P2, I want `loran schema` to return JSON Schema Draft 2020-12 of the data types, so that I can wire Loran into function-calling APIs (Anthropic, OpenAI, Gemini, MCP) without hand-coding the schemas.
- **US-043.** As P2, I want the MCP surface to be read-only and lazy-loaded, so that I can discover capabilities without exhausting my context window or accidentally triggering writes.
- **US-044.** As P2, I want `AI_AGENT=1` to suppress the TUI and force `--format json`, so that I never have to detect or avoid an interactive renderer.

7.6 Sysadmin workflows

- **US-050.** As P4 (sysadmin), I want `loran update` to verify a minisign signature before extracting any content, so that a compromised CDN cannot inject pages.
- **US-051.** As P4, I want `loran update --dry-run` to report exactly what would change without touching disk, so that I can audit updates before applying them at fleet scale.
- **US-052.** As P4, I want per-distro overlays to be authored in their respective distro repos (Bravais overlay in the Bravais repo) and shipped via the same signed tarball pipeline, so that overlay provenance is auditable.

7.7 Accessibility

- **US-060.** As P6 (accessibility-dependent), I want `loran` invoked with `NO_COLOR=1` to produce screen-reader-friendly text without ANSI escapes, so that my reader does not announce escape sequences.
- **US-061.** As P6, I want full keyboard navigation in the TUI via both CUA and Vim bindings, so that I can use whichever scheme matches my muscle memory.
- **US-062.** As P6, I want all colour pairings to meet WCAG 2.1 AA contrast, so that text remains legible at standard zoom levels.

8. Functional Requirements

Each requirement is testable. The implementation is described in `loran-spec-v0_2.md`; this section enumerates what the implementation must satisfy.

8.1 Catalog browsing & discovery

ID	Requirement	Phase
FR-001	The system shall list all catalogued tools with filterable fields <code>(--category)</code> , <code>(--replaces)</code> , <code>(--safe-alias-for)</code> , <code>(--fields)</code> .	Ingot
FR-002	The system shall list all categories with entry counts.	Ingot

ID	Requirement	Phase
FR-003	The system shall perform fuzzy text search across <code>(name)</code> , <code>(summary)</code> , <code>(replaces)</code> , and <code>(tags)</code> fields.	Ingot
FR-004	The system shall perform reverse-lookup: given a legacy tool name, return all catalogued tools that supersede it.	Ingot
FR-005	Reverse-lookup shall distinguish "modern alternative" (broad <code>(replaces)</code>) from "alias-safe replacement" (<code>(safe_alias_for)</code>) via a <code>(--safe-alias)</code> filter flag.	Ingot
FR-006	The system shall present a default TUI view on TTY invocation with no arguments: categories pane + tools pane, fuzzy search bound to <code>(/)</code> , in-app help bound to <code>(?)</code> .	Billet

8.2 Curated content display

ID	Requirement	Phase
FR-010	<code>(loran show <tool>)</code> shall display the Steelbore intro block (when an index entry exists) followed by body content per the resolution chain.	Ingot
FR-011	Body resolution shall be: custom page in any overlay → tldr page if <code>(tldr_page)</code> is set and cached → no-entry diagnostic. Live <code>(--help)</code> is never invoked by <code>(show)</code> .	Ingot
FR-012	When no entry exists, the system shall emit a no-entry diagnostic with the hint <code>(loran new <tool> --edit)</code> and a <code>(see also: loran search ...)</code> line.	Ingot
FR-013	The detail view (TUI and text) shall surface <code>(pairs_with)</code> entries as a "Pairs well with" sidebar or section.	Billet
FR-014	The detail view shall surface <code>(safe_alias_for)</code> entries with an explicit affirmative badge ("safe to alias as X") distinct from the broader <code>(replaces)</code> set.	Billet
FR-015	The detail view shall surface <code>(written_in)</code> with a "  " badge when the value is <code>(rust)</code> .	Billet

8.3 Live --help capture

ID	Requirement	Phase
FR-020	<code>(loran help <tool>)</code> shall resolve the target binary via <code>(\$PATH)</code> only — never trust a user-supplied path.	Ingot
FR-021	The capture shall be performed via direct <code>(execve)</code> -style subprocess invocation: <code>(argv = [tool, "--help"])</code> . No shell, no string interpolation.	Ingot

ID	Requirement	Phase
FR-022	The capture shall enforce a 5-second wall-clock timeout, SIGKILL on overrun, with exit code <code>LIVE_HELP_TIMEOUT = 9</code> .	Ingot
FR-023	On non-zero exit, the capture shall retry the sequence <code>--help → -h → help</code> and prefer the non-empty result.	Ingot
FR-024	The capture shall set <code>PAGER="bat -pp"</code> , <code>MANPAGER="bat -pp"</code> , <code>LESS=""</code> in the subprocess environment, falling back to <code>cat</code> if <code>bat</code> is absent on <code>\$PATH</code> .	Ingot
FR-025	The captured output shall render in a de-themed frame: monochrome chrome, NOT the Steelbore palette, with a <code>LIVE OUTPUT – uncured, captured from <tool> -help at <ISO 8601 UTC></code> header.	Ingot
FR-026	<code>loran help</code> shall never be invocable via the MCP surface (writes-side block + arbitrary-subprocess attack-surface block).	Bloom

8.4 Page authoring

ID	Requirement	Phase
FR-030	<code>loran new <tool></code> shall scaffold a new page from a user-editable template located at <code>\$XDG_DATA_HOME/loran/templates/tool.md</code> .	Billet
FR-031	The default write destination is <code>\$XDG_DATA_HOME/loran/overlays/user/<category>/<tool>.md</code> .	Billet
FR-032	<code>--scope=upstream</code> redirects the write into a user-configured path to a local checkout of the upstream pages tree.	Billet
FR-033	Interactive mode prompts for <code>category</code> with autocomplete from <code>categories.toml</code> , then <code>summary</code> , then <code>replaces</code> , then opens <code>\$EDITOR</code> on the body.	Billet
FR-034	Non-interactive mode accepts every required and optional field as a flag (<code>--category=</code> , <code>--replaces=</code> , <code>--safe-alias-for=</code> , <code>--pairs-with=</code> , <code>--summary=</code> , <code>--no-edit</code>).	Billet
FR-035	<code>loran validate</code> shall validate every page on disk against the frontmatter schema, emit machine-readable output, and exit non-zero on any violation.	Billet
FR-036	Schema validation shall enforce <code>safe_alias_for ⊆ replaces</code> and emit <code>PAGE_PARSE_ERROR = 8</code> with file path and line number on violation.	Billet

8.5 Catalog updates

ID	Requirement	Phase
FR-040	<code>loran update</code> shall fetch the upstream pages tarball over HTTPS from <code>https://Loran.Steelbore.com/pages/v1/pages.tar.gz</code> .	Billet
FR-041	The fetch shall send <code>If-None-Match</code> against the cached ETag; 304 means no work to do.	Billet
FR-042	The fetch shall verify SHA-256 against the manifest before any extraction.	Billet
FR-043	The fetch shall verify a minisign ed25519 signature against a trust-pinned public key (compiled into the binary) before any extraction.	Billet
FR-044	On signature or checksum failure, the system shall exit with <code>TARBALL_VERIFY_FAILED = 11</code> and not modify the working catalog.	Billet
FR-045	Extraction shall be atomic: write to a temp directory, then <code>rename</code> into place.	Billet
FR-046	After successful extraction, the system shall rebuild the postcard index cache.	Billet
FR-047	The same flow shall be supported for the tldr-pages tarball from <code>https://tldr-pages.github.io/assets/tldr.zip</code> , but with SHA-256-only verification (tldr does not sign upstream).	Billet
FR-048	A <code>--require-signatures</code> flag shall refuse the tldr fetch entirely, for security-strict deployments.	Billet
FR-049	<code>--dry-run</code> shall report what would be fetched, verified, and extracted, without touching disk.	Billet

8.6 Overlay management

ID	Requirement	Phase
FR-050	The index loader shall merge three overlay layers in precedence order: upstream <code>pages/</code> → <code>overlays/<active-distro>/</code> → <code>overlays/user/</code> .	Billet
FR-051	Layer merging shall be field-by-field, not record-by-record: a user overlay can override <code>summary</code> without re-stating <code>category</code> or <code>replaces</code> .	Billet

ID	Requirement	Phase
FR-052	The active distro shall be resolved from <code>/etc/os-release</code> (<code>ID=bravais</code>), (<code>ID=ferrite</code>), ...), falling back to "generic" when no match.	Billet
FR-053	The active overlay shall be overridable via <code>config.toml</code> (<code>active_overlay = "bravais"</code>) and the <code>--overlay <name></code> flag.	Billet
FR-054	User overlays may add categories but may not remove upstream categories.	Billet

8.7 Agent surface (JSON, schema, MCP)

ID	Requirement	Phase
FR-060	Every data-returning command shall accept <code>--json</code> and produce output conforming to the SFRS §6 envelope.	Ingot
FR-061	Auto-detection shall switch to <code>--format json</code> when stdout is not a TTY, per SFRS §5 cascade.	Ingot
FR-062	<code>loran schema</code> shall emit JSON Schema Draft 2020-12 of all public data types: page, list-entry, search-result, error envelope.	Bloom
FR-063	<code>loran describe</code> shall emit a self-description manifest per SFRS §4: tool name, version, sub-commands with one-line descriptions, capability tags.	Ingot
FR-064	<code>loran mcp</code> shall run as an MCP server over stdio.	Bloom
FR-065	The MCP surface shall expose only read-only verbs: <code>list</code> , <code>show</code> , <code>find</code> , <code>search</code> , <code>categories</code> . <code>update</code> , <code>new</code> , <code>validate</code> , <code>help</code> shall NOT be exposed.	Bloom
FR-066	MCP <code>tools/list</code> shall advertise tool names + capability tags only; full schemas shall come from <code>tools/get</code> (lazy-loading per <code>steelbore-agentic-cli</code> §6).	Bloom
FR-067	When <code>AI_AGENT=1</code> or <code>AGENT=1</code> is set, the system shall never activate the TUI and shall warn on stderr per SFRS §5.	Ingot
FR-068	Every JSON error envelope shall include a runnable <code>hint</code> field per SFRS tips-thinking discipline.	Ingot

8.8 Self-description ingestion (Phase 3)

ID	Requirement	Phase
FR-070	The index loader shall expose an <code>Ingestor</code> trait abstraction allowing additional content sources to plug in.	Ingot
FR-071	The <code>DescribeIngestor</code> shall invoke <code><tool> describe --json</code> against SFRS-compliant Steelbore binaries on <code>\$PATH</code> and synthesise baseline catalog entries.	Bloom
FR-072	<code>DescribeIngestor</code> results shall be overlaid by any curated page that exists for the same tool — curated content always wins.	Bloom
FR-073	The trust list for <code>DescribeIngestor</code> (which binaries are safe to invoke) shall be defined by an allowlist baked into the upstream pages tarball.	Bloom

8.9 Internationalisation (deferred to v1.x)

ID	Requirement	Phase
FR-080	The page format shall reserve the <code>language</code> frontmatter field for future i18n use.	Ingot
FR-081	When i18n activates, translated pages shall live under <code>pages.<lang>/</code> directories, following tldr-pages precedent.	v1.x
FR-082	The CLI surface (sub-command names, flag descriptions, error messages) remains English-only in v1.	v1.x

9. Non-Functional Requirements

9.1 Performance

ID	Requirement
NFR-001	<code>loran show <known-tool></code> shall complete in <50ms wall-clock on modern hardware with a warm postcard index.
NFR-002	<code>loran list</code> (full catalog, JSON output) shall complete in <100ms for catalogs up to 1,000 entries.
NFR-003	Index rebuild shall complete in <500ms for catalogs up to 500 entries.

ID	Requirement
NFR-004	<code>loran update</code> shall complete in <10s over a typical broadband connection (50 Mbps down), excluding TLS handshake.
NFR-005	TUI input-to-render latency shall be <16ms (60Hz refresh cap) for navigation actions.

9.2 Memory safety

ID	Requirement
NFR-010	Implementation shall be in Rust, governed by the Steelbore Rust Guidelines.
NFR-011	<code>unsafe</code> blocks shall be avoided; any required <code>unsafe</code> shall be confined, documented, and reviewed.
NFR-012	Dependency audit via <code>cargo-audit</code> shall be run before every release; CVEs in dependencies block release.
NFR-013	No C dependencies in the v1 path; all FFI is Rust-Rust via <code>cdylib</code> or pure-Rust alternatives.

9.3 Reliability

ID	Requirement
NFR-020	Tarball extraction shall be atomic — partial updates shall never replace the working catalog.
NFR-021	Index build shall fail loud on schema violations; no silent skipping of malformed pages.
NFR-022	Subprocess invocations (<code>loran help</code>) shall enforce wall-clock timeouts with SIGKILL.
NFR-023	Network failures during <code>loran update</code> shall be retried with exponential backoff up to 3 attempts.

9.4 Security

ID	Requirement
NFR-030	Upstream tarballs shall be minisign-signed with ed25519. Verification is mandatory before extraction.

ID	Requirement
NFR-031	The trust-pinned ed25519 public key shall be compiled into the Loran binary via <code>include_bytes!</code> .
NFR-032	Key rotation shall require a new Loran release; the procedure shall be documented in OPERATIONS.md.
NFR-033	Loran shall never execute upstream content. Pages are inert Markdown.
NFR-034	<code>loran help</code> subprocess invocations shall avoid shell interpretation entirely; argv is constructed directly.
NFR-035	Loran shall never write outside <code>\$XDG_DATA_HOME/loran/</code> , <code>\$XDG_CACHE_HOME/loran/</code> , <code>\$XDG_CONFIG_HOME/loran/</code> , or paths explicitly named by the user via <code>--scope=upstream</code> .

9.5 Privacy (PFA per Standard §7)

ID	Requirement
NFR-040	No telemetry, analytics, crash reporting, or usage tracking, regardless of opt-in/opt-out flags.
NFR-041	All user data (overlays, config, cache, templates) shall be stored locally under XDG paths.
NFR-042	Network access shall be limited to <code>loran update</code> (tarball fetch over HTTPS to known endpoints).
NFR-043	Loran shall ship without a "phone home" mechanism of any kind, including for version checks.

9.6 POSIX compliance & SFRS conformance

ID	Requirement
NFR-050	Default text-mode output shall be parseable with POSIX <code>grep</code> / <code>awk</code> / <code>cut</code> / <code>sed</code> / <code>tr</code> alone.
NFR-051	stdout shall carry data only; stderr shall carry everything else (logs, progress, banners, errors).
NFR-052	UTF-8 without BOM for all output streams.
NFR-053	All timestamps in stored, transmitted, and logged form shall be ISO 8601 UTC with the <code>Z</code> suffix.

ID	Requirement
NFR-054	The <code>--local-time</code> flag is explicitly prohibited per SFRS §1 Rule 1.
NFR-055	Durations shall be represented in ISO 8601 duration form (<code>PT1H30M</code>) in machine-readable output.
NFR-056	All units shall be metric (SI); no AM/PM, no imperial.

9.7 Accessibility

ID	Requirement
NFR-060	All UI surfaces shall meet WCAG 2.1 AA contrast; the Steelbore palette satisfies this by construction.
NFR-061	<code>NO_COLOR=1</code> and <code>FORCE_COLOR=0</code> shall suppress all ANSI escape codes.
NFR-062	The TUI shall support full keyboard navigation via both CUA (Ctrl+C/X/V, arrow keys) and Vim (<code>h j k l</code> , modal) bindings per Standard §8.
NFR-063	The TUI shall provide an explicit accessibility mode that disables non-essential animations and respects reduced-motion preferences.
NFR-064	Text-mode output shall be screen-reader-compatible without further configuration.

9.8 Licensing & attribution

ID	Requirement
NFR-070	Licence: GPL-3.0-or-later for all source files.
NFR-071	SPDX headers (<code>// SPDX-License-Identifier: GPL-3.0-or-later</code>) on all <code>.rs</code> and <code>Cargo.toml</code> files.
NFR-072	<code>LICENSE</code> , <code>NOTICE.md</code> , <code>CONTRIBUTING.md</code> , <code>README.md</code> present at repo root from the first commit per Standard v1.1 §5.2.
NFR-073	Attribution per Standard §13.2 in <code>--version</code> , <code>--help</code> footer, README, and TUI About: maintainer name (<code>Mohamed Hammad</code>), contact (<code>Mohamed.Hammad@Steelbore.com</code>), project URL (<code>https://Loran.Steelbore.com/</code>).

9.9 Compatibility

ID	Requirement
NFR-080	Tier 1 platforms: Linux x86_64 (glibc and musl), Linux aarch64, FreeBSD amd64.
NFR-081	Tier 2 platforms: macOS arm64 (development convenience for the maintainer; not a primary target).
NFR-082	Minimum Rust version (MSRV): the latest stable at release time. Loran does not pin old stable.
NFR-083	Terminal compatibility: any terminal supporting truecolor or 256-color and Unicode box-drawing characters.

10. Product Surface Overview

A high-level view; the full command surface, flag inventory, and exact resolution chains are specified in §4 and §7 of `loran-spec-v0_2.md`.

10.1 Sub-commands at a glance

Command	Purpose
<code>loran</code>	TUI if TTY, else <code>loran list --json</code> (auto-detection per SFRS §5)
<code>loran list</code>	List catalogued tools, filterable
<code>loran show <tool></code>	Show resolved curated page (Steelbore intro + body, curated-or-fail)
<code>loran help <tool></code>	Capture and render <code><tool> --help</code> directly (always-live, de-themed)
<code>loran find <legacy></code>	Reverse lookup: what supersedes <code><legacy></code> ?
<code>loran search <q></code>	Fuzzy search across name/summary/replaces/tags
<code>loran categories</code>	List categories with entry counts
<code>loran new <tool></code>	Scaffold a new curated page
<code>loran update</code>	Refresh upstream + tldr tarballs (signature-verified)
<code>loran validate</code>	Validate all pages against the frontmatter schema

Command	Purpose
<code>loran schema</code>	Emit JSON Schema of own data types
<code>loran describe</code>	Self-description manifest for agents
<code>loran mcp</code>	Run as read-only MCP server over stdio (Phase 3)

10.2 Three output modes

- **TUI mode** (default on TTY): ratatui-based dual-pane browser, Steelbore palette, Vim + CUA keybindings.
- **Text mode** (no TTY, `--format text`, or stdout redirected): POSIX-parseable plain text.
- **JSON mode** (`--json`, `--format json`, agent env vars set, or stdout piped): SFRS-compliant envelope.

10.3 Three audience layers

- **Human users** interact via TUI and `--help` text.
- **Scripts and pipelines** interact via the JSON envelope and stable exit codes.
- **AI agents** interact via the JSON envelope, `loran schema`, `loran describe`, and the MCP server.

11. Page Authoring Workflow

11.1 Page format (high-level)

A Loran page is a single Markdown file with TOML frontmatter. Required fields: `name`, `category`, `summary`. Optional fields include `replaces`, `safe_alias_for`, `pairs_with`, `official`, `tldr_page`, `tags`, `written_in`, `since`, `aliases`. Full schema in spec §6.1.

11.2 Authoring loop

1. Curator runs ``loran new <tool>``.
2. Loran prompts for category (autocomplete) and summary.
3. Loran writes a scaffolded file under `overlays/user/` (or upstream tree if `--scope=upstream`).
4. `$EDITOR` opens on the body.
5. Curator writes Steelbore-flavoured Markdown.
6. Curator saves and closes.
7. Curator runs ``loran validate`` to check schema.
8. If errors: fix and re-validate.
9. If clean: page is live in the local catalog immediately.
10. (Optional) Curator opens a PR upstream via the user-side git checkout.

11.3 Style guidance (informative, not normative)

A well-written Loran page should:

- Lead with a one-line summary of what the tool does in Steelbore-canonical terms.
- Explicitly call out Steelbore-specific aliases, environment variables, or shell-profile integrations.
- Include 3–5 example invocations that exercise the most common use cases.
- Cross-reference companion tools via `(pairs_with)`.
- Be honest about `(safe_alias_for)` — listing a tool there is a Steelbore endorsement that scripts won't break.

These are conventions, not validation rules. The schema is what `(loran validate)` enforces.

12. Distribution & Trust Model

12.1 Publisher pipeline (out of scope for v1 PRD, but constrains v1)

The upstream pages tarball is produced by a publisher pipeline (separate project, not part of the Loran binary) that:

1. Aggregates pages from this repo's `(pages/)` tree and from per-distro repos (Bravais, Ferrite OS, etc.).
2. Validates every page against the frontmatter schema.
3. Builds a tarball `((pages.tar.gz))`.
4. Generates a manifest with version, ETag, SHA-256.
5. Signs the tarball with the publisher's ed25519 minisign key.
6. Publishes manifest + tarball + signature to `(https://Loran.Steelebore.com/pages/v1/)`.

12.2 Client verification chain

On the Loran client side, `(loran update)` performs:

1. Manifest fetch (ETag-aware).
2. Tarball fetch (only if manifest changed).
3. Signature fetch `((pages.tar.gz.minisig))`.
4. SHA-256 verification against the manifest.
5. Minisign signature verification against the trust-pinned public key compiled into the Loran binary.
6. Atomic extraction.
7. Index rebuild.

Any failure between steps 1 and 5 aborts the update with `TARBALL_VERIFY_FAILED = 11` and leaves the existing catalog untouched.

12.3 Key rotation

The ed25519 public key is baked into the Loran binary at compile time. Rotation therefore requires a new Loran release. This is a deliberate trade-off: it makes key compromise harder to recover from gracefully (no over-the-wire key rotation), but it also makes key spoofing impossible without compromising the binary distribution chain.

Specific rotation procedure, parallel-key transition windows, and emergency-rotation policy are deferred to v0.3 (Open Question 1).

12.4 The tldr-pages asymmetry

The tldr-pages upstream project does not currently sign its tarballs. Loran handles this honestly:

- The tldr tarball is verified by SHA-256 only.
- A `--require-signatures` flag refuses the tldr fetch entirely.
- This asymmetry is documented in the spec, the PRD, and the user-facing `--help` text for `loran update`.

13. Steelbore Ecosystem Integration

Loran does not live in a vacuum. Its integrations with other Steelbore projects are first-class requirements.

13.1 Distro overlays

Distro	Overlay location	Source-of-truth
Bravais	<code>overlays/bravais/...</code>	The Bravais project repository
Ferrite OS	<code>overlays/ferrite/...</code>	The Ferrite OS project repository
(generic)	<code>pages/</code> (no overlay)	The Loran repository

Each per-distro overlay is authored and maintained in its respective project repo, then surfaced into Loran via the upstream publisher tarball pipeline. This keeps distro opinions co-located with their distros.

13.2 SFRS describe ingestion (Phase 3)

Every Steelbore CLI must implement `<tool> describe --json` per SFRS §4. The `DescribeIngestor` (Phase 3) invokes this on Steelbore-native binaries and synthesises baseline catalog entries. Net effect: every new Steelbore CLI (Ferrocast, Caliper, Craton, Ironway, Zamak,

Flux, Mawaqit, and future projects) gets a Loran entry for free, with curated pages overlaid on top where they exist.

13.3 Cross-CLI references

Other Steelbore CLIs should reference Loran in their `--help` output:

```
USAGE:
  flux <COMMAND>

Steelbore reference: loran show flux
```

This creates a network effect — Loran becomes the gravitational centre of Steelbore tool discovery without other CLIs having to embed catalog content directly.

13.4 Bravais shell-profile integration

The Bravais Nushell profile ships with aliases that match Loran's `safe_alias_for` recommendations. Loran does not author or manage the shell profile, but it does serve as the canonical reference for which aliases are safe — meaning the Bravais profile and Loran's catalog must agree at release time.

14. Phasing & Release Plan

Per Steelbore Standard §2, release codenames follow the cast-form list.

14.1 Phase 1: Ingot (v0.1.x → v0.x — text-mode usable)

Outcome: A useful binary that lists, shows, finds, and searches the bundled tool catalog, with full JSON output and SFRS-compliant flags. No network, no overlays, no TUI.

Scope:

- Cargo workspace + posture files (README, NOTICE, CONTRIBUTING, LICENSE) per Standard v1.1 §5.2.
- All global flags per SFRS §3.
- Sub-commands: `list`, `show`, `help`, `find`, `search`, `categories`, `describe`, `schema` (placeholder, full schema in Bloom).
- Page format parser (`loran-pages` crate).
- `Ingestor` trait abstraction in `loran-index` (only `MarkdownPagesIngestor` implemented).
- Index from bundled `pages/` tree (built into binary).
- JSON envelope per SFRS §6.
- POSIX-compliant text mode.
- Live `--help` capture with full sandbox per FR-020 through FR-025.

- Agent env-var detection (`(AI_AGENT, AGENT, CI, CLAUDECODE, CURSOR_AGENT, GEMINI_CLI)`).
- Attribution per Standard §13.2.

Out of Ingot: TUI, tarball update, overlays, page authoring, MCP, signature verification.

Exit criteria: All Ingot-tagged FRs (FR-001 to FR-005, FR-010 to FR-012, FR-020 to FR-025, FR-060, FR-061, FR-063, FR-067, FR-068, FR-070, FR-080) pass. NFRs in §9.1, §9.2, §9.4, §9.5, §9.6, §9.8 met for the implemented surface.

14.2 Phase 2: Billet (v1.0 — the user-visible milestone)

Outcome: The product as users will know it. Full TUI, tarball updates with signature verification, per-distro and per-user overlays, page authoring, schema validation.

Scope:

- TUI (`(loran-tui)` crate): dual-pane browser, fuzzy search, detail view, Vim + CUA bindings.
- Tarball update (`(loran-tldr)` crate handles both upstream pages and tldr).
- Minisign verification (`(minisign-verify)` crate).
- Overlay loader with three-layer merge.
- `(loran new)` + interactive scaffolding.
- `(loran validate)` with schema enforcement.
- Exit code `(TARBALL_VERIFY_FAILED = 11)` and the full tips-thinking error catalog per spec §12.3.
- `(--require-signatures)` flag for security-strict deployments.

Exit criteria: All Ingot + Billet FRs pass. The Bravais default install includes a populated Loran catalog and a Nushell profile aligned with `(safe_alias_for)` data. NFRs in §9.3, §9.7 fully met.

14.3 Phase 3: Bloom (v1.x — the agentic completion)

Outcome: Loran becomes the primary tool-discovery surface for AI agents on Steelbore systems, and the ecosystem becomes self-documenting via SFRS describe ingestion.

Scope:

- `(loran-mcp)` crate: read-only MCP server over stdio per SFRS, with lazy-loading discipline.
- `(loran schema)` emits full JSON Schema Draft 2020-12 for Anthropic, OpenAI, Gemini, MCP function-calling.
- `(DescribeIngestor)` implementation: ingests `(<tool> describe --json)` from allowlisted Steelbore binaries.
- Cross-distro overlay surfacing (the Bravais and Ferrite OS overlays are part of the upstream tarball, not just the per-distro overlays).
- Minisign key rotation procedure documented and tested (resolving Open Question 1).

Exit criteria: All FRs and NFRs pass. Loran can be invoked by Claude Code, Codex CLI, and Cursor without TUI activation, with no special configuration beyond the standard agent env vars.

14.4 Release cadence

Personal-hobby pace per Standard §5.1. No service-level commitments. Semantic versioning applies — breaking changes to the JSON schema or page format require a major version bump and a deprecation cycle.

15. Success Metrics

Personal-hobby projects don't have MAU/DAU metrics. The right success metrics here are about *coverage* and *quality*, not adoption volume.

15.1 Catalog metrics

- **M-01: Curated coverage.** Percentage of tools in the default Bravais install that have a Loran-curated page (not just a tldr fallback). Target: $\geq 80\%$ at Billet release.
- **M-02: Reverse-lookup coverage.** Percentage of common legacy tools (`ls`, `cat`, `grep`, `find`, `ps`, `top`, `cp`, `mv`, `du`, `df`, `dig`, `curl`, `wget`, `awk`, `sed`, `tar`, `head`, `tail`, `less`, `more`) that have at least one entry in some tool's `replaces` field. Target: 100% by Billet release.
- **M-03: Pairing coverage.** Average number of `pairs_with` entries per curated page. Target: ≥ 1.5 average across the catalog at Billet release.

15.2 Quality metrics

- **M-04: Schema strictness.** Percentage of pages in the upstream tarball that pass `loran validate` on a clean checkout. Target: 100% always; CI blocks any tarball publish where validation fails.
- **M-05: Cold response time.** P95 cold-cache latency for `loran show <known-tool>`. Target: $< 50\text{ms}$.
- **M-06: Build-and-bench gate.** Every release passes a benchmark suite confirming NFR-001 through NFR-005.

15.3 Ecosystem-integration metrics

- **M-07: SFRS describe parity.** Number of Steelbore CLIs whose `<tool> describe --json` output is correctly ingested by `DescribeIngestor`. Target: all Steelbore CLIs by Bloom release.
- **M-08: Cross-CLI references.** Number of Steelbore CLIs whose `--help` text references `loran show <self>`. Target: all Steelbore CLIs by Bloom release.

15.4 Anti-metrics (things we deliberately do not measure)

- DAU/MAU. Loran is not a service; how often individual users invoke it is none of our business.
- Install count. PFA precludes phone-home; we have no idea how many users we have and that's fine.
- Page-view counts on `loran show`. Same reason.

16. Dependencies

16.1 Steelbore-internal

Dependency	Type	Relationship
Steelbore Standard v1.1	Specification	Authoritative for naming, palette, fonts, time, attribution, posture
Steelbore SFRS v1.0.0	Specification	Authoritative for CLI shape, JSON envelope, exit codes, agent env vars
<code>rust-guidelines</code> skill	Implementation	Loaded at every Rust-writing session
Bravais project	Content	Provides the Bravais overlay; ships Loran in default install
Ferrite OS project	Content	Provides the Ferrite overlay; ships Loran in default install
Other Steelbore CLIs	Integration	Reference <code>loran show <self></code> in <code>--help</code> ; expose <code><tool> describe --json</code>

16.2 Third-party Rust crates

Full canonical list in spec §3.3. Critical ones:

Crate	Purpose	Failure mode if missing/broken
<code>clap</code> (derive)	CLI parsing	Blocker — no project without it
<code>serde</code> + <code>serde_json</code> + <code>toml</code>	Serialisation	Blocker
<code>pulldown-cmark</code>	Markdown parsing	Blocker — pages are Markdown
<code>ratatui</code> + <code>crossterm</code>	TUI	Blocker for Billet TUI
<code>jiff</code>	Time / timestamps	Blocker per Standard §12.5
<code>ureq</code> + <code>rustls</code>	HTTPS for tarball fetch	Blocker for Billet update
<code>minisign-verify</code>	Signature verification	Blocker for Billet trust model
<code>nucleo-matcher</code>	Fuzzy search	Could be substituted; not load-bearing
<code>postcard</code>	Index cache format	Could be substituted; not load-bearing
<code>rmcp</code>	MCP server	Blocker for Bloom MCP

16.3 External services

Service	Purpose	SLA expectations
<code>https://Loran.Steelbore.com/</code>	Hosts upstream tarball + manifest + signature	Hobby pace; no SLA
<code>https://tldr-pages.github.io/</code>	tldr-pages upstream tarball	External; outside our control

16.4 No-dependency commitments

- No async runtime in the v1 fast path (no `tokio` in `loran-cli`, `loran-core`, `loran-index`, `loran-pages`, `loran-render`, `loran-tldr`, `loran-tui`). Async, if it appears, is confined to `loran-mcp`.
- No C dependencies. All FFI is Rust-Rust.
- No JavaScript or web runtime of any kind.

17. Risks & Mitigations

17.1 Curation burden

Risk: Authoring high-quality Steelbore-curated pages for ~150 tools is a substantial ongoing investment. Risk that the catalog grows but quality decays.

Mitigations:

- Schema validation (`(loran validate)`) blocks malformed pages.
- A documented style guide for pages (covered briefly in spec §11.3; expanded in CONTRIBUTING.md).
- Fallback chain (Steelbore intro → custom → tldr → no-entry) means an incomplete catalog still produces useful output: the absence of a curated body falls back to tldr automatically.
- `(DescribeIngestor)` (Bloom) auto-generates baseline pages from SFRS `(describe)`, reducing the from-zero authoring cost.

17.2 Upstream signing key compromise

Risk: If the ed25519 publisher key is compromised, an attacker can sign malicious tarballs.

Mitigations:

- Key stored on a hardware token in normal operation; published key is widely-known.
- Rotation requires a new Loran release; users on old Loran versions cannot be tricked by a rotated key from a newer publisher.
- `(--require-signatures)` blocks the tldr asymmetry from being a sideways attack vector.
- Open Question 1 (key rotation policy) will define emergency-rotation procedures.

17.3 tldr-pages upstream changes

Risk: tldr-pages may change its tarball format, distribution URL, or licensing.

Mitigations:

- Loran depends on tldr only as a fallback; a working catalog is possible without tldr.
- `(tldr_page)` is an optional frontmatter field; pages can be self-sufficient.
- A failed tldr fetch is non-fatal (logs warning, continues without tldr).
- If tldr is permanently unavailable, the chain degrades gracefully: custom page or no-entry diagnostic.

17.4 Catalog drift from actual distro inventory

Risk: Bravais ships tool X, but X isn't in Loran's catalog (or vice versa).

Mitigations:

- Per-distro overlays in distro repos make it easy to add distro-specific entries.

- `DescribeIngestor` (Bloom) automatically picks up any SFRS-compliant Steelbore binary on `$PATH`.
- Drift detection: an `xtask` or CI check compares the Bravais default-install package list against the Loran catalog and flags gaps.

17.5 Agent-misinterpretation of `safe_alias_for`

Risk: An AI agent reads `safe_alias_for = ["cat"]` on a page for `bat`, and aggressively recommends `alias cat=bat` in the user's shell rc — but the user's existing scripts use a `cat`-specific edge case `bat` doesn't preserve.

Mitigations:

- `safe_alias_for` is intentionally conservative: entries are vetted by curators to be safe in the common case.
- `replaces` is the broader (less safe) field; agents are expected to recommend with caveats when the entry is only in `replaces`.
- Curation guidance (in CONTRIBUTING.md) emphasises that listing in `safe_alias_for` is a strong endorsement, not a soft recommendation.

17.6 Scope creep into general-purpose man-page rendering

Risk: Users (or AI agents) request features that would turn Loran into a general-purpose docs renderer (man pages, info pages, online docs, etc.).

Mitigations:

- Non-goals (§6) are explicit: Loran renders its own pages and captures live `--help`. Nothing else.
- The verb split (`show` vs `help`) is the load-bearing scope boundary.

17.7 Single-maintainer bus factor

Risk: Personal-hobby project; maintainer absence stalls everything.

Mitigations:

- License (GPL-3.0-or-later) permits forks.
- CONTRIBUTING.md documents the path for contributors.
- Posture is explicit (Standard §5.1): hobby pace, no SLA, fork-encouraged.

18. Open Questions

The same set as spec §15. Resolution belongs in v0.3 of the spec; the PRD inherits the same open list.

1. **Minisign key rotation policy.** Acceptable cadence? Emergency-rotation procedure? Parallel-key transition windows?
2. **pairs_with reciprocity.** Should `loran validate` warn on non-reciprocal pairings, or accept asymmetry as intentional?
3. **DescribeIngestor trust list.** Allowlist baked into the upstream tarball (current leaning)? Self-declaration via SFRS `describe`? Cryptographic attestation?

None of these blocks Phase 1 (Ingot) or Phase 2 (Billet) implementation. All three are Phase 3 (Bloom) concerns or future-policy concerns.

19. Steelbore Standard v1.1 Compliance Audit

Per Standard §14, every Steelbore artifact must pass this audit. This section is the canonical sign-off for Loran's design.

§	Requirement	Status	Notes
2	Metallurgical naming	⚠	Deviation acknowledged. "Loran" is a heritage engineering acronym (LORAN, radio navigation), not metallurgical. Granted by §5.4 maintainer discretion. Release codenames (Ingot/Billet/Bloom) and internal crate names align with the convention.
3.1	Memory safety	✓	100% Rust per NFR-010. <code>rust-guidelines</code> skill loaded at every implementation session.
3.2	Concurrency designed-in; benchmarking	✓	Async confined to <code>loran-mcp</code> . Benchmarks gate every release per M-06.
3.3	Hardened security; PQC	✓	minisign + rustls for trust + transport. PQC posture: ed25519 today; hybrid scheme revisit when stable.
4	GPL-3.0-or-later + SPDX	✓	NFR-070, NFR-071.
5.2	Required posture files (README/NOTICE/CONTRIBUTING/LICENSE)	✓	NFR-072.
5.1	Default personal-hobby posture	✓	Stated in README Project Posture section.
6.1	POSIX-compliant CLI	✓	NFR-050, NFR-051. Per SFRS §1 Rule 3.
7	PFA: no tracking, minimal perms, local	✓	NFR-040 through NFR-043.
8	CUA + Vim bindings	✓	NFR-062. Both schemes in TUI.
9	Steelbore palette; Void Navy bg	✓	Curated content uses palette tokens; <code>loran help</code> capture frame uses monochrome (intentional brand boundary).
10	FOSS fonts	N/A	Terminal app; uses user's terminal font. Docs use Share Tech Mono / Inconsolata per Standard.

§	Requirement	Status	Notes
11	Material Design / WCAG 2.1 AA	Partial	No GUI in v1 → Material Design N/A. WCAG AA contrast met by palette construction. Accessibility per §9.7.
12	ISO 8601 / UTC / Z-suffix / 24h / metric	✓	NFR-053 through NFR-056. All timestamps in JSON envelope and live-help captures carry <code>Z</code> suffix.
13	Attribution	✓	NFR-073. Surfaced in <code>--version</code> , <code>--help</code> , README, TUI About.

The single 🚩 on §2 is a known and accepted deviation, documented openly so it remains visible to future reviewers rather than disappearing into the maintainer's head.

20. References & Related Documents

20.1 Loran-internal

- `loran-spec-v0_2.md` — Canonical technical specification. Read first for any implementation work.
- `README.md` — User-facing project introduction (will be authored at first commit).
- `NOTICE.md` — No-warranty / no-liability statement (Standard v1.1 §5.2).
- `CONTRIBUTING.md` — Contribution scope and process (Standard v1.1 §5.2).
- `AGENTS.md` — Agent-generic context for AI coding tools.
- `CLAUDE.md` — Claude Code-specific context.
- `SKILL.md` — Loran's own capability surface for the Steelbore Skills system.

20.2 Steelbore standards

- The Steelbore Standard v1.1** — Naming, priorities, license, posture, platform, PFA, key bindings, palette, fonts, UI/UX, time, attribution.
- Steelbore SFRS v1.0.0** — Dual-Mode Self-Documenting CLI Framework. Authoritative for `--json`, `--format`, exit codes, JSON envelope, agent env vars, MCP threshold rule.
- Steelbore Agentic-CLI Standard** — Lazy-loading discipline for MCP, `AGENTS.md` / `CLAUDE.md` conventions, tips-thinking error hints.
- Steelbore Rust Guidelines** — Crate choices (`jiff`, `thiserror` + `anyhow`, `tracing`), `unsafe` policy, error handling.

20.3 External references

- tldr-pages project** — `https://tldr.sh/` — Loran consumes the tldr tarball as a fallback

content source.

- **minisign** — <https://jedisct1.github.io/minisign/> — Signature format used for upstream tarball verification.
- **XDG Base Directory Specification** — <https://specifications.freedesktop.org/basedir-spec/> — Determines [\\$XDG_DATA_HOME](#), [\\$XDG_CACHE_HOME](#), [\\$XDG_CONFIG_HOME](#).
- **CommonMark** — <https://commonmark.org/> — Markdown dialect used for page bodies.
- **JSON Schema Draft 2020-12** — <https://json-schema.org/draft/2020-12/json-schema-core.html> — Schema dialect emitted by [loran schema](#).
- **Model Context Protocol (MCP)** — <https://modelcontextprotocol.io/> — Agent server protocol used by [loran mcp](#).

Forged in Steelbore.